Python Engineering - Reference Guide

Class 10 - Advanced Functions: argumentos y funciones como objetos

Guia sencilla y algo ampliada para entender los tipos de argumentos que puede recibir una funcion y como Python permite pasar funciones como datos.

1. Parametro vs argumento

Un **parametro** es el nombre escrito en la definicion de la funcion. Un **argumento** es el valor real que entregamos al llamar a la funcion.

```
def energy(mass, velocity):   # mass, velocity = parameters
    ...

energy(10, 50)                # 10, 50 = arguments
```

2. Argumentos posicionales

Que hacen: asignan valores segun el orden en que se pasan.

Que reciben: valores separados por comas en la llamada.

Que producen: cada valor queda asociado al parametro de la misma posicion.

```
def initial_energy(h0, v0, g):
    ...

initial_energy(0.0, 50.0, 9.81)
# h0=0.0, v0=50.0, g=9.81
```

Son compactos, pero el orden debe ser correcto.

3. Keyword arguments

Que hacen: asignan valores por nombre en vez de por posicion.

Que reciben: pares nombre=valor.

Que producen: el valor se asigna al parametro cuyo nombre coincide.

```
initial_energy(
    v0=50.0,
    g=9.81,
    h0=0.0
)
```

El orden deja de ser importante si los nombres son correctos.

4. Default arguments

Que hacen: proporcionan un valor por defecto si el usuario no entrega ese argumento.

Que reciben: se definen como parametro=valor.

Que producen: la funcion puede llamarse omitiendo ese argumento.

```
def initial_energy(h0, v0, g=9.81):
    ...

initial_energy(0.0, 50.0)
# g uses 9.81

initial_energy(0.0, 50.0, g=1.62)
# g is overridden
```

Regla importante: normalmente los parametros sin valor por defecto se colocan antes que los parametros con valor por defecto.

5. *args en la DEFINICION

Que hace: recoge un numero variable de argumentos posicionales.

Que recibe: cero, uno o muchos valores posicionales.

Que devuelve internamente: dentro de la funcion los agrupa en una **tupla**.

```
def average_measurements(*values):
    print(type(values))
    print(values)

average_measurements(10, 20, 30)

# <class 'tuple'>
# (10, 20, 30)
```

El nombre args no es obligatorio. Lo importante es el asterisco *. Por convencion suele escribirse *args.

6. **kwargs en la DEFINICION

Que hace: recoge un numero variable de argumentos nombrados.

Que recibe: pares nombre=valor.

Que devuelve internamente: un **diccionario**.

```
def summary(**kwargs):
    print(type(kwargs))
    print(kwargs)

summary(h=120, v=-10)

# <class 'dict'>
# {'h': 120, 'v': -10}
```

7. * al LLAMAR una funcion: unpacking

Que hace: desempaqueta una secuencia y entrega sus elementos como argumentos posicionales.

Que recibe: normalmente list, tuple u otra secuencia iterable.

Que produce: una llamada equivalente a escribir cada elemento por separado.

```
values = [10, 20, 30]

average_measurements(*values)

# equivalent to:
average_measurements(10, 20, 30)
```

8. ** al LLAMAR una funcion: dictionary unpacking

Que hace: desempaqueta un diccionario en argumentos nombrados.

Que recibe: un diccionario cuyas claves son strings.

Que produce: una llamada keyword; las claves deben coincidir con los nombres esperados por la funcion.

```
config = {
    'h0': 0.0,
    'v0': 50.0,
    'g': 9.81
}

initial_energy(**config)

# equivalent to:
initial_energy(h0=0.0, v0=50.0, g=9.81)
```

9. Definir vs llamar: misma sintaxis, funcion distinta

Sintaxis	En def	En llamada
*name	recoge muchos argumentos posicionales en tuple	desempaqueta una secuencia
**name	recoge muchos argumentos keyword en dict	desempaqueta un diccionario

10. Pasar una funcion como argumento

En Python una funcion es tambien un objeto. Puede asignarse a una variable, guardarse o pasarse a otra funcion.

```
def add(a, b):
    return a + b

def apply_operation(operation, x, y):
    return operation(x, y)

apply_operation(add, 5, 3)
# 8
```

Que recibe apply_operation? operation recibe la propia funcion add; x e y reciben numeros. Dentro de apply_operation, operation(x,y) ejecuta la funcion recibida.

11. Funcion vs llamada de funcion

```
add      # function object
add()    # call the function now
```

Cuando hacemos apply_operation(add, 5, 3), usamos add sin parentesis porque queremos entregar la funcion. Si escribieramos add(...), la ejecutariamos antes de pasar el resultado.

12. Por que esto importa en ingenieria numerica?

Muchos algoritmos numericos reciben una funcion matematica como entrada. La idea sera:

```
solve_root(function, ...)
integrate(function, ...)
solve_ode(function, ...)
```

El algoritmo numerico no necesita saber de antemano la ecuacion concreta: recibe la funcion y la evalua cuando la necesita.

13. Resumen rapido

Concepto	Entrada	Resultado / comportamiento
Positional	valores	asignacion por orden
Keyword	nombre=valor	asignacion por nombre
Default	parametro=valor por defecto	permite omitir el argumento
*args	muchos posicionales	tuple dentro de la funcion
**kwargs	muchos keywords	dict dentro de la funcion
*sequence	lista/tupla	desempaqueta posicionalmente
**dictionary	dict	desempaqueta por nombre
function as argument	funcion sin ()	otra funcion puede ejecutarla